SpecBridge: An Agentic Assistant for Turning Jira Tickets into Codebase-Grounded UAT Test Cases

Unmanned Rooks | Hussain Lohawala | Qasim Naik | Tarun Rajai | October 2026

1. Problem Statement and Motivation

Product managers (PMs) describe frontend features in Jira tickets using product language ("let users save a draft"), while engineers must translate this into concrete changes across several large repositories (often micro-frontends). The gap between *what the PM means* and *what the engineer builds* is filled by back-and-forth comments, grooming meetings and rework. PMs rarely know which existing flows a change touches, so edge cases (errors, permissions, empty states) surface late.

- **Stakeholders:** PMs, frontend engineers and QA.
- **Why it matters:** misunderstood requirements are a leading cause of rework and delayed releases; a shared, precise specification *before* implementation reduces both.

2. Existing Approaches and Limitations

- **Manual specs and grooming meetings:** effective but slow, depend on individual knowledge of the codebase, and quickly go stale.
- **Ticket/Confluence templates:** enforce structure (acceptance criteria, user stories) but not correctness; they cannot tell the PM which screens or flows are affected.
- **General-purpose AI writing assistants:** expand or summarise ticket text but are not grounded in the codebase, so they produce generic or wrong references to screens and flows.
- **Code-search and coding assistants:** grounded in code, but aimed at engineers *after* work starts, not at the PM–engineer hand-off.

Opportunity: combine ticket intent with the *current* codebase to produce UAT test cases before coding starts.

3. Objectives and Scope

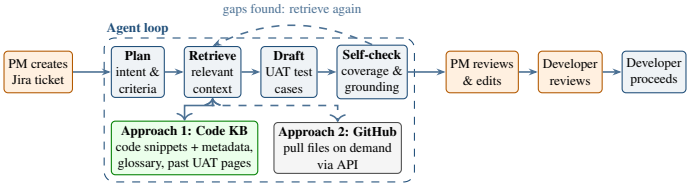
Objective: when a PM creates a Jira ticket, the agent—*before any code changes are made*—writes a Confluence page of UAT test cases describing how the feature should behave: for each case, preconditions, steps (*Action*) and *Expected Result*, covering happy and negative paths, plus the affected screens and open questions where the ticket is unclear. *Actual result* and *pass/fail* are left for testers. The PM reviews and edits the page to match their expectations; the developer then reviews it and uses it as direction for implementation. This removes most back-and-forth and saves time for the whole team.

Assumptions: each agent instance is *team-specific*: it has read access to that team's repositories and its Jira project, and write access only to its Confluence space and ticket comments.

In scope: React/TypeScript frontends across the team's repos; Jira, Confluence and GitHub integration; a code knowledge base; human approval at every hand-off.

Out of scope: writing or modifying code; executable/automated tests; backend/API design; running the application; auto-approving pages or assigning work.

4. Proposed Direction (Agentic Approach)



Context engineering decides what code the agent sees, since whole repos cannot fit in an LLM's context window. We evaluate two approaches:

- **Approach 1 (primary) – Code knowledge base:** an offline indexer splits the team's repos into code snippets at component/function level, each stored with metadata (repo, file path, route, component name) and an embedding, alongside a glossary of product terms and past approved UAT pages. It is refreshed incrementally on each merge. At run time the agent searches the KB and receives only the most relevant snippets. Fast and token-efficient, but needs an index to build and keep fresh.
- **Approach 2 (alternative) – Live pull from GitHub:** the agent searches and fetches files through the GitHub API at run time. Always up to date with no index to maintain, but harder: the agent must navigate large repos without a map, needing more API calls (rate limits), more tokens and more time.

Why agentic: which code is relevant is not known in advance. The LLM must *decide* what to look up next based on what it finds, judge when it has enough evidence, check its test cases against every acceptance criterion and the code (so it cannot invent screens), and turn ambiguity into questions for the PM. Nothing reaches the developer without PM approval.

5. Success and Quality Attributes

- **Test-case coverage:** percentage of Jira acceptance criteria covered by at least one generated UAT case, including negative paths.
- **Grounding:** percentage of referenced screens, components and flows that can be verified in the current codebase, measuring the system's tendency to hallucinate.
- **Usefulness and time saved:** percentage of generated test cases retained by PMs without edits, developer clarity rating (1–5), and change in PM–developer clarification comments and time per ticket.
- **Efficiency:** measure draft-generation time, token usage and cost per ticket. Evaluate whether retrieval and prompt optimisations reduce unnecessary context and computation.
- **Reliability and security:** verify team-scoped, least-privilege access; ensure secrets are not exposed to the model; and confirm that repeated runs update the existing UAT page correctly.